

Notice Technique

Méthode de construction du diagramme de Voronoï:

La création du diagramme est partie d'un raisonnement assez simple, une image est un tableau d'abscisse et d'ordonnée.

Il fallait accueillir la structure qui allait recevoir le diagramme de voronoï. Pour cela nous avons utilisé la librairie PIL. Une image vide à été créée.

Mais cette image est composée de de pleins de pixels et c'est de là que nous sommes partis. Il est bon de savoir qu'un pixel a des coordonnées précises qui dans ce cas la correspondent à une abscisse et une ordonnée.

A partir de là nous avons pu parcourir tous les pixels de l'image et vérifier lequel des points présent dans le fichier txt (donné dans le sujet), celui qui contient les points de voronoï est le plus proche des autres points.

Tous les points présents dans le fichier txt se voit attribuer une couleur au hasard.

Et lorsqu'un pixel lambda se retrouve le plus proche d'un des points présents dans le fichier txt il prend sa couleur. Et c'est là que le voronoï se forme.

Mais pour calculer les distances entre les points pour déterminer le point le plus proche nous avons utiliser une formule trouver après des recherches:
Le carré de la distance entre un point de coordonnées (x,y) et un autre de coordonnées (a,b) est $(x-a)^2 + (y-b)^2$

Suite à tout cela il reste un point à aborder. Comment faire si les coordonnées du point lambda sont exactement les mêmes que celles dans le fichier txt. Cela peut arriver étant donné que l'on parcours vraiment tous les points de l'image, c'est du brute force pure. Il a fallu ajouter une condition pour cela, si le point est comparé avec lui-même il garde sa couleur.

Choix techniques:

Pour ce qui est du langage nous avons choisi le python pour plusieurs raisons. Tout d'abord suite à la ressource en programmation multimédia que nous avons eu cette année nous savons qu'il permet de générer des images et d'utiliser de nombreuses librairies numériques. Comme la librairie PIL qui nous a permis de créer des images ou de colorier des pixels. On a aussi choisi la bibliothèque math pour nous faciliter dans le développement de nos fonctions. Pour pouvoir colorier les cellules du diagramme avec des couleurs différentes nous avons utilisé la bibliothèque random.

Ensuite pour l'interface graphique nous avons fait appel à la librairie Tkinter parce qu'elle est assez simple à prendre en main et on trouve beaucoup de documentation sur Internet.

Architecture:

L'architecture est assez simple fichier points.txt nous permet d'entrer les coordonnées des différents points du diagramme. Le fichier voronoi_cheick.py c'est là où se trouve toute la logique et les calculs qui nous permette de générer le diagramme. Le fichier placer_point.png c'est juste le diagramme qu'on sauvegarde. Enfin le fichier [gui.py](#) c'est l'interface graphique qui appelle le fichier voronoi_cheick.py et affiche le diagramme sur une fenêtre grâce à un bouton.

Limite:

Le code ne prend pas en compte les nombres de type float ce qui limite un peu la précision du placement des points. En de ça plus la taille de l'image est grande et plus il y a de points dans le fichier txt, plus le code prend du temps à s'exécuter. Parce que la stratégie de développement utilisée est la brute force.

L'une des limites est aussi qu'il n'y a pas de ligne de séparation entre les cellules, on passe d'une couleur à une autre sans bordure. Concernant l'interface graphique l'une des limites est qu'il n'y a pas vraiment de gestion d'erreur. Si le fichier point.txt est absent le programme va planter sans qu'on sache pourquoi puisqu'il n'y a pas de message d'erreur.